

Ricerca Operativa

Diego Stefanini — prof. Stefano Novellani, a.a. 2026-27

Indice

1	Tecniche di modellazione	3
1.1	Il problema della fonderia (mix di produzione)	3
1.2	Variabili binarie: zaino e relazioni logiche	5
1.3	Scegliere lati di un grafo: albero di copertura e commesso viaggiatore	7
1.4	Assegnamento, semi-assegnamento e colorazione di grafi	9
1.5	Selezione di sottoinsiemi: partizione, copertura, riempimento	12
1.6	Problemi di flusso su rete	18

1 Tecniche di modellazione

Un modello ha sempre tre pezzi: **variabili** (le decisioni), **funzione obiettivo** (cosa minimizzo o massimizzo), **vincoli** (cosa devo rispettare).

Da sapere — equazione lineare in più variabili (aggiunto, non spiegato in aula)

Un'espressione è **lineare** se ogni variabile compare **da sola, alla prima potenza, moltiplicata solo per un numero**, e i termini si sommano:

$$a_1x_1 + a_2x_2 + \dots + a_nx_n = b$$

con $a_1, \dots, a_n, b$ numeri fissi. Vale lo stesso per $\leq$ e $\geq$ (diseguazioni lineari).

$$4x_1 + x_2 + 0.6x_3 \geq 3250$$

✓ lineare

$$x_1 + 3 \geq 2x_2 - 7$$

✓ lineare (sposto tutto a sinistra)

$$x_1 \cdot x_2 \leq 5$$

✗ prodotto di due variabili

$$x_1^2 + x_2 \leq 5$$

✗ potenza

$$\frac{x_1}{x_1 + x_2} \geq 0.2$$

✗ divisione per variabili (ma si può rendere lineare, vedi sotto)

Perché conta: se funzione obiettivo e vincoli sono lineari, il problema è di **Programmazione Lineare** (PL) e ci sono algoritmi efficienti per risolverlo.

Cosa non si può scrivere in un modello (errori che all'esame escono sempre):

- solo $\leq$, $\geq$ e $=$: niente disuguaglianze strette ($<$, $>$) e niente $\neq$ (« $x \neq y$ » non si può scrivere);
- niente prodotti o divisioni fra variabili, niente valore assoluto $|x|$, niente min o max di variabili dentro i vincoli: solo espressioni lineari.

«s.t.» sotto la funzione obiettivo sta per *subject to*, «soggetto a»: da lì in poi ci sono i vincoli. Ricordarsi sempre di scrivere anche il **dominio** delle variabili ($x \geq 0$, $x \in \{0, 1\} \dots$).

Lo stesso problema si può scrivere con **modelli diversi**, alcuni più efficienti di altri. In questo corso l'efficienza non conta: un modello che funziona è accettabile, anche con qualche vincolo in più non necessario.

1.1 Il problema della fonderia (mix di produzione)

x_1 kg di materiale 1	4% Si · 0,45% Mn · 0,025 €/kg
x_2 kg di materiale 2	1% Si · 0,50% Mn · 0,030 €/kg
x_3 kg di materiale 3	0,6% Si · 0,40% Mn · 0,018 €/kg
x_4 kg di manganese puro	100% Mn · 10 €/kg



1000 pezzi da 1 kg
silicio fra 3,25% e 5,5%
manganese almeno 0,45%

Variabili: x_1, x_2, x_3 = kg di materiale ferroso 1, 2, 3 nel mix; x_4 = kg di manganese puro.

Da percentuali a vincoli: il silicio nel mix, in kg, è $0.04x_1 + 0.01x_2 + 0.006x_3$. Deve essere almeno il 3,25% di 1000 kg, cioè 32,5 kg. Moltiplico tutto per 1000 per togliere le virgole:

$$\begin{aligned} 0.04x_1 + 0.01x_2 + 0.006x_3 &\geq 32.5 \\ 40x_1 + 10x_2 + 6x_3 &\geq 32500 \quad (\times 1000) \end{aligned}$$

Stesso ragionamento per il manganese: $0.0045x_1 + 0.005x_2 + 0.004x_3 + x_4 \geq 4.5$, moltiplicato per 1000.

$$\begin{aligned}
\min z &= 0.025x_1 + 0.030x_2 + 0.018x_3 + 10x_4 \\
x_1 + x_2 + x_3 + x_4 &= 1000 && \leftarrow \text{minimizzo il costo} \\
40x_1 + 10x_2 + 6x_3 &\geq 32500 && \leftarrow \text{totale da produrre} \\
40x_1 + 10x_2 + 6x_3 &\leq 55000 && \leftarrow \text{silicio minimo} \\
45x_1 + 50x_2 + 40x_3 + 10000x_4 &\geq 45000 && \leftarrow \text{silicio massimo} \\
&&& \leftarrow \text{manganese minimo} \\
x_1, x_2, x_3, x_4 &\geq 0 && \leftarrow \text{non negatività}
\end{aligned}$$

Il modello si può scrivere in **forma generale**: al posto dei numeri metto dei **parametri**: $n = 4$ materiali, $K = 1000$ kg, c_i costo al kg, s_i e m_i percentuali di silicio e manganese del materiale i , $\underline{s}$ e $\bar{s}$ silicio minimo e massimo, $\underline{m}$ manganese minimo.

$$\min z = \sum_{i=1}^n c_i x_i \quad \text{s.t.} \quad \sum_{i=1}^n x_i = K, \quad \underline{s}K \leq \sum_{i=1}^{n-1} s_i x_i \leq \bar{s}K, \quad \sum_{i=1}^n m_i x_i \geq \underline{m}K, \quad x_i \geq 0$$

La somma del silicio arriva a $n - 1$: il manganese puro (materiale 4) non contiene silicio.

Un'**istanza** è una configurazione concreta del problema: il modello generale con dei valori al posto dei parametri. Quello con i numeri sopra è un'istanza.

Un vincolo che capita spesso è quello **in percentuale**:

«Il materiale 1 deve essere almeno il 20% dei materiali ferrosi usati.»

$$\frac{x_1}{x_1 + x_2 + x_3} \geq 0.2$$

non lineare: diviso per variabili

moltiplico per
il denominatore $\rightarrow$

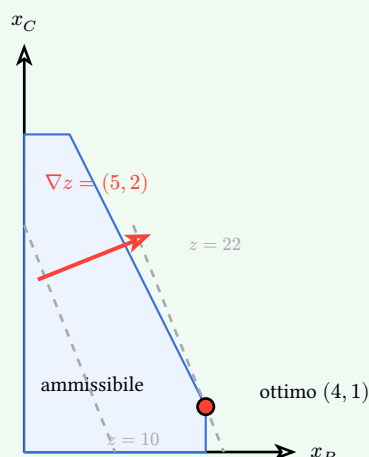
$$x_1 \geq 0.2(x_1 + x_2 + x_3)$$

lineare

Si può moltiplicare senza girare il $\geq$ perché il denominatore è una somma di quantità ≥ 0 . Lo stesso trucco serve quando chiedono una **media pesata**.

Da sapere — gradiente (aggiunto, non spiegato in aula)

Per una funzione lineare $z = c_1 x_1 + c_2 x_2$ il **gradiente** è il vettore dei coefficienti, $\nabla z = (c_1, c_2)$. Indica la direzione in cui z **cresce più in fretta**.

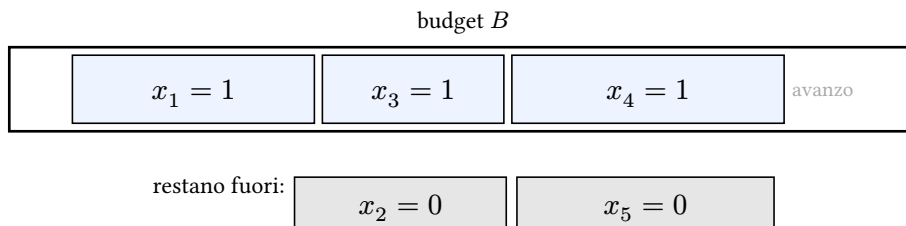


Esempio: $\max z = 5x_P + 2x_C$ con $x_P \leq 4$, $x_C \leq 7$, $2x_P + x_C \leq 9$.

- Le rette tratteggiate sono **linee di livello**: punti con lo stesso z . Sono perpendicolari al gradiente.
- Per un **max** sposto la retta nella direzione del gradiente finché tocca ancora la regione: l'ultimo punto toccato è l'ottimo, $(4, 1)$ con $z = 22$.
- Per un **min** vado nella direzione opposta, $-\nabla z$.

1.2 Variabili binarie: zaino e relazioni logiche

Un investitore ha un capitale B e n progetti. Il progetto i costa c_i e rende w_i . Quali progetti finanzia per rendere il più possibile senza sforare il budget?



$$x_i = \begin{cases} 1 & \text{se finanzia il progetto } i \\ 0 & \text{altrimenti} \end{cases}$$

$$\max z = \sum_{i=1}^n w_i x_i$$

$$\text{s.t. } \sum_{i=1}^n c_i x_i \leq B$$

$$x_i \in \{0, 1\} \quad \forall i \in I$$

Le variabili **binarie** (o booleane) servono per le decisioni sì/no.

Il trucco: moltiplicando per x_i , nella somma entrano **solo i progetti scelti** ($x_i = 1$); gli altri valgono 0.

$\sum c_i x_i \leq B$ è il **vincolo di budget**.

Con le variabili binarie si scrivono anche le **relazioni logiche** fra le decisioni: «se finanzia questo devo finanziare quello», «al massimo uno fra questi». Ogni relazione diventa un vincolo lineare. Tutte le variabili qui sono binarie: $x_A = 1$ se finanzia il progetto A. Nelle tabelle, ✓ = combinazione permessa dal vincolo.

Sono esempi separati: non devono valere tutti insieme.

Il caso più semplice: **esattamente, almeno o al massimo uno** fra A e B.

x_A	x_B	$x_A + x_B = 1$	$x_A + x_B \geq 1$	$x_A + x_B \leq 1$
0	0	✗	✗	✓
0	1	✓	✓	✓
1	0	✓	✓	✓
1	1	✗	✓	✗

1. **esattamente uno** fra A e B: $x_A + x_B = 1$

2. **almeno uno**: $x_A + x_B \geq 1$

3. **al massimo uno**: $x_A + x_B \leq 1$

Funziona con più variabili e un termine noto diverso: «non più di due fra A, B, C» è $x_A + x_B + x_C \leq 2$.

Negazione: B = non A si scrive $x_B = 1 - x_A$ (è il caso 1).

Implicazione: se finanzia A, allora devo finanziare anche C.

x_A	x_C	$x_A \leq x_C$
0	0	✓
0	1	✓
1	0	✗
1	1	✓

4. «Se investo in A devo investire in C; se non investo in A, C è libero»: $x_A \leq x_C$.

Se $x_A = 1$ il vincolo forza $x_C = 1$; se $x_A = 0$ diventa $0 \leq x_C$, sempre vero.

Il «se e solo se» (relazione biunivoca) è l'uguaglianza: $x_A = x_C$.

And: si vuole $C = A \wedge B$.

x_A	x_B	x_C	5	6	5 + 6
0	0	0	✓	✓	✓
0	0	1	✓	✗	✗
0	1	0	✓	✓	✓
0	1	1	✓	✗	✗
1	0	0	✓	✓	✓
1	0	1	✓	✗	✗
1	1	0	✗	✓	✗
1	1	1	✓	✓	✓

5. «Se finanzia **sia A che B** devo finanziare C; altrimenti C è libero»:

$$x_C \geq x_A + x_B - 1$$

(a destra c'è 1 solo se $x_A = x_B = 1$, altrimenti ≤ 0)

6. «Posso finanziare C **solo se** finanzia A e B contemporaneamente»:

$$x_C \leq x_A \text{ e } x_C \leq x_B$$

Insieme danno $C = A \wedge B$: la colonna 5 + 6 è ✓ solo dove $x_C = x_A \cdot x_B$.

Con tre progetti: «se finanzia A, B e C devo finanziare D» è $x_D \geq x_A + x_B + x_C - 2$. In generale con k progetti il -1 diventa $-(k-1)$.

I due vincoli del 6 si possono sommare in uno solo: $2x_C \leq x_A + x_B$, cioè $x_C \leq \frac{x_A + x_B}{2}$. Se almeno uno fra A e B vale 0, a destra c'è al massimo $\frac{1}{2}$, e una variabile binaria $\leq \frac{1}{2}$ può solo valere 0. Stessa cosa per il 7 più avanti: $2x_C \geq x_A + x_B$. Per l'esame le due scritture valgono uguale; in pratica i vincoli separati sono in genere più forti.

Or: si vuole $C = A \vee B$.

x_A	x_B	x_C	7	8	7 + 8
0	0	0	✓	✓	✓
0	0	1	✓	✗	✗
0	1	0	✗	✓	✗
0	1	1	✓	✓	✓
1	0	0	✗	✓	✗
1	0	1	✓	✓	✓
1	1	0	✗	✓	✗
1	1	1	✓	✓	✓

7. «**Devo** finanziare C se finanzia almeno uno fra A e B»:

$$x_C \geq x_A \text{ e } x_C \geq x_B$$

8. «**Posso** finanziare C solo se finanzia almeno uno fra A e B»:

$$x_C \leq x_A + x_B$$

Insieme danno $C = A \vee B$.

Or esclusivo: si vuole C finanziato quando c'è esattamente uno fra A e B.

x_A	x_B	x_C	9	10	9 + 10
0	0	0	✓	✓	✓
0	0	1	✓	✗	✗
0	1	0	✗	✓	✗
0	1	1	✓	✓	✓
1	0	0	✗	✓	✗
1	0	1	✓	✓	✓
1	1	0	✓	✓	✓
1	1	1	✓	✗	✗

9. «**Devo** finanziare C se finanzia esattamente uno fra A e B»:

$$x_C \geq x_A - x_B \text{ e } x_C \geq x_B - x_A$$

10. «**Posso** finanziare C solo se finanzia esattamente uno fra A e B»:

$$x_C \leq 2 - x_A - x_B \text{ e } x_C \leq x_A + x_B$$

Insieme danno $C = A \oplus B$.

Trappola sul 9: viene da scrivere $x_C \geq x_A + x_B$. Ma con A e B entrambi finanziati diventa $x_C \geq 2$, che una binaria non può rispettare: il problema diventa **inammissibile** (nessuna soluzione rispetta

tutti i vincoli). Il testo del 9 dice solo quando C è obbligatorio; con A e B entrambi a 1 non dice niente, quindi C deve restare libero.

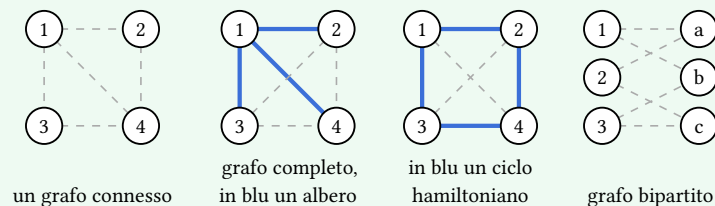
Schema che si ripete: il «**devo**» (se ... allora C) è un vincolo $x_C \geq \dots$ che spinge C a 1; il «**posso solo se**» è un vincolo $x_C \leq \dots$ che tiene C a 0.

1.3 Scegliere lati di un grafo: albero di copertura e commesso viaggiatore

Nello zaino le variabili binarie sceglievano un **sottoinsieme** dei progetti. È il loro uso generale: le variabili binarie selezionano sottoinsiemi di un insieme dato e, con i vincoli giusti, il sottoinsieme scelto può avere una struttura complessa quanto si vuole. Qui l'insieme è quello dei **lati di un grafo**, e la struttura richiesta è prima un albero, poi un ciclo.

Da sapere — grafi (aggiunto, non spiegato in aula)

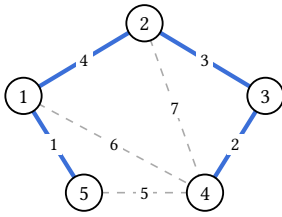
Un **grafo non orientato** $G = (V, E)$ è un insieme di **vertici** V (i pallini) e di **lati** E (le linee). Un lato unisce due vertici e si scrive $\{i, j\}$: le graffe dicono che l'ordine non conta, $\{i, j\}$ e $\{j, i\}$ sono lo stesso lato. Con $n = |V|$ si indica il numero di vertici ($|\cdot|$ = numero di elementi di un insieme). Nei grafi **orientati** le linee hanno un verso e si chiamano **archi**, (i, j) ; qui sono non orientati e si chiamano **lati** (se il prof dice «arco», qui leggi «lato»).



- **connesso**: da ogni vertice si arriva a ogni altro camminando sui lati.
- **ciclo**: un giro di lati che torna al vertice di partenza. Un ciclo **hamiltoniano** passa per **tutti** i vertici, **una sola volta** ciascuno.
- **albero**: grafo connesso e **senza cicli**. Un albero su n vertici ha sempre $n - 1$ lati. Un **albero di copertura** di G è un albero fatto con lati di G che tocca tutti i vertici.
- **completo**: c'è un lato fra ogni coppia di vertici.
- **bipartito**: i vertici si dividono in due gruppi e ogni lato va da un gruppo all'altro, mai dentro lo stesso gruppo.
- **taglio**: preso un sottoinsieme S di vertici, $V \setminus S$ sono i vertici **fuori** da S ; i lati del taglio sono quelli con un estremo in S e l'altro in $V \setminus S$, cioè quelli che «escono» da S .

Albero di copertura di costo minimo (Minimum Spanning Tree, MST). Il grafo $G = (V, E)$ è connesso e non orientato: i vertici $V = \{1, \dots, n\}$ sono città, i lati sono i collegamenti che si possono costruire, e il lato $\{i, j\}$ costa $c_{ij} > 0$. Si vuole che ogni città raggiunga ogni altra attraverso la rete, spendendo il meno possibile.

Variabili: una per lato,



$$x_{ij} = \begin{cases} 1 & \text{se costruisco il collegamento } \{i, j\} \in E \\ 0 & \text{altrimenti} \end{cases}$$

Scegliere i valori delle x vuol dire scegliere un insieme di lati. Si potrebbe anche usare una variabile per ogni albero possibile, ma gli alberi sono un numero esponenziale: meglio costruire la soluzione **a pezzettini**, un lato alla volta. Nel disegno i lati in blu hanno $x_{ij} = 1$: collegano tutte le città con costo $1 + 4 + 3 + 2 = 10$, il minimo possibile per questo grafo.

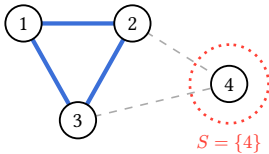
Funzione obiettivo: come nello zaino, nella somma entrano solo i lati scelti,

$$\min z = \sum_{\{i,j\} \in E} c_{ij} x_{ij}$$

Vincoli: come si scrive «la rete è connessa»? Si guarda il problema da fuori. Se la rete è connessa, qualunque gruppo di città S prenda, almeno un lato scelto deve **uscire** da S , sennò le città di S restano isolate dal resto. Questo vale per ogni S che non sia vuoto e non sia tutto V (se $S = V$ non c'è nessun «resto» da raggiungere). Si scrive $S \subset V$, sottoinsieme **proprio** cioè diverso da V , e $S \neq \emptyset$:

$$\sum_{i \in S, j \in V \setminus S} x_{ij} \geq 1 \quad \forall S \subset V, S \neq \emptyset \quad x_{ij} \in \{0, 1\} \quad \forall \{i, j\} \in E$$

La somma conta i lati scelti che attraversano il taglio: deve essere almeno 1.



Esempio: scelgo i lati del triangolo 1-2-3. Il vertice 4 resta fuori dalla rete. Con $S = \{4\}$ i lati che escono da S sono $\{2, 4\}$ e $\{3, 4\}$, entrambi con $x = 0$: la somma fa $0 < 1$, il vincolo è violato e la soluzione è esclusa.

Il **numero di vincoli è esponenziale**: uno per ogni sottoinsieme S . I sottoinsiemi di n vertici sono 2^n ; tolti $\emptyset$ e V restano $2^n - 2$ vincoli. Con 30 città sono già più di un miliardo.

Il modello chiede solo che la rete sia **connessa**, non che sia un albero. Ma all'ottimo è per forza un albero: se i lati scelti formassero un ciclo, togliendo un lato del ciclo la rete resterebbe connessa (le città si raggiungono dall'altra parte del ciclo) e costerebbe meno, perché ogni $c_{ij} > 0$. Quindi una soluzione con un ciclo non può essere ottima.

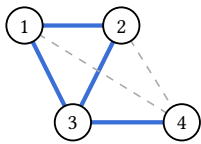
Un'altra idea: un albero su n vertici ha sempre $n - 1$ lati, quindi si potrebbe scrivere $\sum_{\{i,j\} \in E} x_{ij} = n - 1$.

Da solo non basta: $n - 1$ lati possono chiudere un ciclo e lasciare fuori un vertice (è proprio il triangolo di prima: 3 lati, $n - 1 = 3$, ma il 4 è isolato). **Insieme ai vincoli di taglio è ridondante:** non serve, perché con costi positivi l'ottimo è già un albero. Diventerebbe necessario se i costi potessero essere negativi: allora al modello converrebbe prendere lati in più, e bisognerebbe fermarlo a $n - 1$.

Problema del commesso viaggiatore (Travelling Salesman Problem, TSP). Il grafo $G = (V, E)$ è **completo** e non orientato: $V = \{1, \dots, n\}$ sono città, $E = \{\{i, j\} : i, j \in V, i \neq j\}$ contiene tutti i collegamenti possibili, ognuno con costo $c_{ij} \geq 0$. Si cerca un **ciclo hamiltoniano di costo minimo**: parto da una città, visito ciascuna delle altre **una e una sola volta**, torno alla città di partenza, e il giro costa il meno possibile.

Variabili e funzione obiettivo sono le stesse dell'albero: $x_{ij} = 1$ se uso il collegamento $\{i, j\}$, e $\min z = \sum_{\{i,j\} \in E} c_{ij} x_{ij}$. Cambiano i vincoli.

Senza vincoli il min non sceglierebbe nessun lato (costo 0). Prima idea: in un ciclo che passa per tutte le città ci sono tanti lati quanti vertici, quindi $\sum_{\{i,j\} \in E} x_{ij} = n$. Non basta:

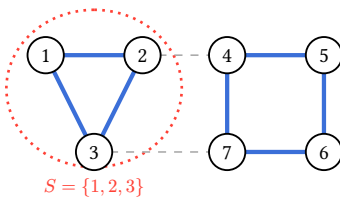


4 lati su 4 vertici, e il grafo è anche connesso, ma non è un giro: il vertice 3 tocca **tre** lati scelti, il 4 uno solo.

Nel giro invece in ogni città **entro una volta ed esco una volta**: ogni vertice tocca esattamente **due** lati scelti. Fissato il vertice i , sommo le x dei lati che lo toccano:

$$\sum_{j:\{i,j\} \in E} x_{ij} = 2 \quad \forall i \in V$$

Questo però non basta. I lati in blu qui sotto danno a ogni vertice esattamente due lati, eppure non sono **un giro** ma **due** cicli separati (sottocicli):



Ogni vertice ha grado 2, ma il commesso viaggiatore non passa mai dal triangolo al quadrato.

Il rimedio sono i **vincoli di taglio dell'albero**: con $S = \{1, 2, 3\}$ nessun lato scelto esce da S ($\{2, 4\}$ e $\{3, 7\}$ hanno $x = 0$), quindi il vincolo è violato. Se il grafo è connesso e ogni vertice ha grado 2, l'unica possibilità è un solo ciclo che passa per tutti: un ciclo hamiltoniano.

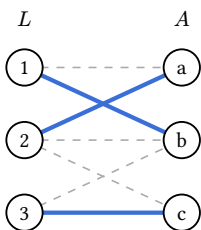
$$\begin{aligned} \min z &= \sum_{\{i,j\} \in E} c_{ij} x_{ij} && \leftarrow \text{costo del giro} \\ \sum_{j:\{i,j\} \in E} x_{ij} &= 2 \quad \forall i \in V && \leftarrow \text{entro ed esco da ogni città} \\ \sum_{i \in S, j \in V \setminus S} x_{ij} &\geq 1 \quad \forall S \subset V, S \neq \emptyset && \leftarrow \text{niente sottocicli (connessione)} \\ x_{ij} &\in \{0, 1\} \quad \forall \{i, j\} \in E \end{aligned}$$

Il vincolo $\sum x_{ij} = n$ ora è **ridondante**: se ogni vertice tocca 2 lati, sommando su tutti i vertici conto $2n$, e ogni lato l'ho contato due volte (una per estremo), quindi i lati sono n . Nella somma sui lati invece $\{i, j\}$ compare una volta sola: il lato non ha verso, $\{i, j\}$ e $\{j, i\}$ sono lo stesso.

È il modello dell'albero di copertura più i vincoli di grado 2. Non è l'unico modello del TSP: si può usare una variabile per ogni ciclo hamiltoniano (ma sono un numero esponenziale e bisogna generarli tutti), oppure sostituire i vincoli di taglio con vincoli che vietano direttamente i sottocicli.

1.4 Assegnamento, semi-assegnamento e colorazione di grafi

Un'azienda deve assegnare delle attività a dei lavoratori. Lavoratori L e attività A sono due insiemi con lo stesso numero n di elementi. Assegnare il lavoratore $i \in L$ all'attività $j \in A$ costa c_{ij} . Tutte le attività vanno assegnate e ogni lavoratore riceve **una e una sola** attività; si vuole il costo minimo.



Si può vedere come una scelta di lati sul grafo **bipartito** $G = (L \cup A, E)$: a sinistra i lavoratori, a destra le attività, un lato per ogni coppia possibile. Non tutti sanno fare tutto (il lavoratore 1 non sa fare c): l'importante è che un assegnamento completo esista.

Un assegnamento è un insieme di lati in cui ogni vertice, di qualunque lato, tocca **esattamente un** lato scelto (in blu: 1 fa b, 2 fa a, 3 fa c).

$$x_{ij} = \begin{cases} 1 & \text{se assegno il lavoratore } i \in L \text{ all'attività } j \in A \\ 0 & \text{altrimenti} \end{cases}$$

$$\begin{aligned}
\min z &= \sum_{i \in L} \sum_{j \in A} c_{ij} x_{ij} && \leftarrow \text{costo totale} \\
\sum_{j \in A} x_{ij} &= 1 \quad \forall i \in L && \leftarrow \text{ogni lavoratore ha una sola attività} \\
\sum_{i \in L} x_{ij} &= 1 \quad \forall j \in A && \leftarrow \text{ogni attività ha un solo lavoratore} \\
x_{ij} &\in \{0, 1\} \quad \forall i \in L, j \in A
\end{aligned}$$

Ogni riga è il vincolo «esattamente uno fra...» delle relazioni logiche ($x_A + x_B = 1$). Servono **tutte e due**: con solo «ogni attività ha un lavoratore» potrei dare due attività allo stesso lavoratore e lasciarne un altro senza niente; con solo l'altra, due lavoratori potrebbero fare la stessa attività. Insieme si chiamano **vincoli di assegnamento**. L'assegnamento è un **problema polinomiale**.

Da sapere — problema polinomiale (aggiunto, non spiegato in aula)

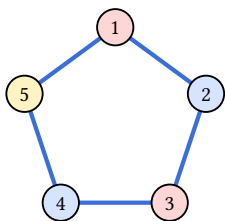
Un problema è **polinomiale** se esiste un algoritmo che lo risolve in un tempo che cresce come una potenza della dimensione ($n^2, n^3, \dots$) e non come 2^n . In pratica: si risolve in fretta anche quando n è grande. Non è ovvio guardando il modello: il modello elenca i vincoli, l'algoritmo è un'altra cosa.

Gli stessi vincoli tornano nel commesso viaggiatore su un grafo **orientato**, dove il lato (i, j) va da i a j e non è lo stesso di (j, i) . Il «grado 2» si spezza in due vincoli simili a quelli dell'assegnamento: da ogni città **esco** una volta, $\sum_j x_{ij} = 1$, e in ogni città **entro** una volta, $\sum_i x_{ij} = 1$.

Nell'assegnamento il vincolo «esattamente uno» vale **dai due lati**: ogni lavoratore ha un'attività e ogni attività ha un lavoratore. Se vale da **un lato solo** si parla di **vincoli di semi-assegnamento**: ogni elemento del primo insieme riceve esattamente un elemento del secondo, ma un elemento del secondo può essere dato a molti (o a nessuno).

Assegnamento di frequenze. Una compagnia telefonica copre la città con un insieme di antenne $S = \{1, \dots, n\}$, tutte attive. A ogni antenna va assegnata una frequenza fra quelle disponibili $F = \{f_1, \dots, f_m\}$. Due antenne troppo vicine non possono avere la stessa frequenza, altrimenti interferiscono. Ogni frequenza ha un costo (lo stesso per tutte), quindi si vuole coprire la città usando il **minor numero di frequenze**.

L'interferenza si disegna con il **grafo di incompatibilità** $G = (S, E)$: i vertici sono le antenne e c'è il lato $\{i, j\} \in E$ se le antenne $i \neq j$ sono troppo vicine.



Ogni frequenza è un **colore**: **rosso** f_1 , **blu** f_2 , **giallo** f_3 . Due antenne unite da un lato devono avere colori diversi. Le antenne 1 e 3 non sono vicine e possono condividere f_1 : è il semi-assegnamento, una frequenza serve più antenne.

Qui servono 3 frequenze: con due sole, alternandole lungo il giro 1-2-3-4-5, la 5 e la 1 (vicine) avrebbero la stessa.

Variabili: due famiglie, una per decidere **chi usa cosa**, una per sapere **quali frequenze sono usate**:

$$x_{if} = \begin{cases} 1 & \text{se assegno la frequenza } f \in F \text{ all'antenna } i \in S \\ 0 & \text{altrimenti} \end{cases} \quad y_f = \begin{cases} 1 & \text{se uso la frequenza } f \in F \\ 0 & \text{altrimenti} \end{cases}$$

$$\begin{aligned}
\min z &= \sum_{f \in F} y_f && \leftarrow \text{numero di frequenze usate} \\
\sum_{f \in F} x_{if} &= 1 \quad i \in S && \leftarrow \text{ogni antenna ha una sola frequenza} \\
x_{if} + x_{jf} &\leq 1 \quad \{i, j\} \in E, f \in F && \leftarrow \text{antenne vicine: frequenze diverse} \\
y_f &\geq x_{if} \quad i \in S, f \in F && \leftarrow \text{una frequenza assegnata è una frequenza usata} \\
x_{if} &\in \{0, 1\} \quad i \in S, f \in F \\
y_f &\in \{0, 1\} \quad f \in F
\end{aligned}$$

Ogni vincolo è una relazione logica già vista:

- $\sum_f x_{if} = 1$ è «**esattamente uno**»: il semi-assegnamento (per ogni antenna, non per ogni frequenza);
- $x_{if} + x_{jf} \leq 1$ è «**al massimo uno**»: fra due antenne vicine i e j al massimo una usa f . Si scrive per ogni lato del grafo di incompatibilità e per ogni frequenza.
- $y_f \geq x_{if}$ è il **legame** fra le due famiglie di variabili. È l'implicazione «se assegno f all'antenna i , allora f è usata» ($x_{if} \leq y_f$). Se $x_{if} = 1$ il vincolo forza $y_f = 1$. Se nessuna antenna usa f il vincolo dice solo $y_f \geq 0$ e y_f sarebbe libera: ma la funzione obiettivo è un min, quindi y_f va a 0 da sola.

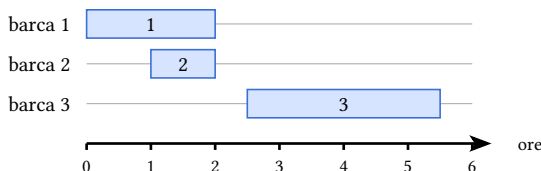
La funzione obiettivo conta le frequenze usate: y è un «vettorino» di 0 e 1 lungo $|F|$, e la sua somma è il numero di 1.

Senza quest'ultimo vincolo il modello sarebbe sbagliato: niente lega le y alle x , quindi il min metterebbe tutte le y_f a 0 (costo 0) mentre le antenne usano comunque le frequenze.

Ogni volta che un modello ha **due o più famiglie di variabili** serve un vincolo che le leghi. Il significato che do a una variabile quando la definisco a parole (« y_f vale 1 se uso la frequenza») è come un commento nel codice: il solutore non lo legge. Quel significato deve essere imposto dai vincoli.

Questo modello è la formulazione di un problema classico, la **colorazione di un grafo** (graph coloring): dare un colore a ogni vertice, usando il minor numero di colori, in modo che due vertici uniti da un lato abbiano colori diversi. Lo stesso modello risolve problemi che sembrano non avere niente in comune con le antenne, come il prossimo.

Moli e barche (ordinamento di lavori su macchine). Un porto ha un insieme di moli $M = \{1, \dots, k\}$, tutti uguali. Ogni giorno arrivano n barche, $B = \{1, \dots, n\}$. La barca i arriva all'istante $t_i \geq 0$ e occupa il molo per una durata d_i per scaricare, quindi riparte a $t_i + d_i$. Le barche non possono aspettare: vanno servite appena arrivano. Ogni barca va assegnata a **esattamente un molo**, e due barche non possono stare sullo stesso molo in intervalli di tempo che si sovrappongono. Si vuole usare il **minor numero di moli**.



Esempio con tre barche: la 1 sta in porto nell'intervallo $[0, 2]$, la 2 in $[1, 2]$, la 3 in $[2.5, 5.5]$.

La 1 e la 2 si sovrappongono (quando arriva la 2, la 1 è ancora al molo): non possono condividere un molo. La 3 arriva quando le altre due sono già ripartite: può usare il molo di una delle due. Bastano 2 moli.

Viene da pensare a una variabile per ogni molo e per ogni istante di tempo («il molo m è occupato all'istante t »). Non conviene: gli istanti sono infiniti, e per usarli bisognerebbe spezzare il tempo in intervallini (**discretizzare**). Ma soprattutto **il tempo qui non è una decisione**: arrivo e durata sono dati, la barca i occupa il molo da t_i a $t_i + d_i$ qualunque cosa io scelga. Una variabile serve per ciò che devo decidere; ciò

che so già è un dato. (Se potessi scegliere **quando** servire una barca, allora sì, servirebbe una variabile per il tempo.)

Il tempo si usa **prima** di scrivere il modello, sui dati, per costruire il **grafo di incompatibilità** $G = (B, E)$: i vertici sono le barche, e c'è un lato fra due barche quando i loro intervalli hanno almeno un istante in comune,

$$\{i, j\} \in E \quad \text{se} \quad [t_i, t_i + d_i] \cap [t_j, t_j + d_j] \neq \emptyset$$

($\cap$ è l'intersezione, $\emptyset$ l'insieme vuoto: «l'intersezione non è vuota»). Nell'esempio E ha un solo lato, $\{1, 2\}$.

Le decisioni sono due, come per le antenne: **su quale molo va ogni barca e quali moli uso**.

$$x_{im} = \begin{cases} 1 & \text{se assegno la barca } i \in B \text{ al molo } m \in M \\ 0 & \text{altrimenti} \end{cases} \quad y_m = \begin{cases} 1 & \text{se uso il molo } m \in M \\ 0 & \text{altrimenti} \end{cases}$$

$$\begin{aligned} \min z &= \sum_{m \in M} y_m && \leftarrow \text{numero di moli usati} \\ \sum_{m \in M} x_{im} &= 1 \quad i \in B && \leftarrow \text{ogni barca ha un solo molo} \\ x_{im} + x_{jm} &\leq 1 \quad \{i, j\} \in E, m \in M && \leftarrow \text{barche incompatibili: moli diversi} \\ y_m &\geq x_{im} \quad i \in B, m \in M && \leftarrow \text{un molo con una barca è un molo usato} \\ x_{im} &\in \{0, 1\} \quad i \in B, m \in M \\ y_m &\in \{0, 1\} \quad m \in M \end{aligned}$$

È lo stesso modello dell'assegnamento di frequenze, con altri nomi:

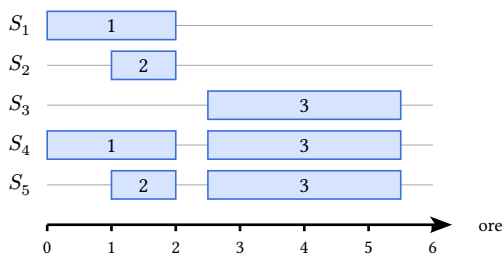
	frequenze	moli e barche	lavori su macchine
vertici del grafo	antenne	barche	lavori
colori	frequenze	moli	macchine
lato = incompatibili	troppo vicine	intervalli sovrapposti	lavori nello stesso momento

Per questo il problema si chiama in generale **ordinamento di lavori su macchine**: le macchine sono i moli, i lavori sono le barche. Avevamo visto che lo stesso problema si può scrivere con modelli diversi; qui succede il contrario, **lo stesso modello risolve problemi diversi**.

1.5 Selezione di sottoinsiemi: partizione, copertura, riempimento

Nel modello dei moli la soluzione è costruita a pezzettini: una variabile per ogni coppia barca-molo. Niente vieta di usare **pezzi più grandi**. Invece di decidere barca per barca, elenco tutti i **turni** possibili di un molo (cioè tutti i gruppi di barche che un molo può servire in una giornata senza sovrapposizioni) e uso una variabile per ogni turno: lo scelgo oppure no.

Sia $\mathcal{F}$ la famiglia di tutti i sottoinsiemi $S \subseteq B$ di barche che **non si sovrappongono fra loro**, e che quindi possono stare tutte sullo stesso molo. Ogni $S \in \mathcal{F}$ è un possibile turno (o schedulazione) di un molo. $\mathcal{F}$ è un «insieme di insiemi». Con le tre barche dell'esempio i turni possibili sono 5:



$$\mathcal{F} = \{\{1\}, \{2\}, \{3\}, \{1,3\}, \{2,3\}\}$$

Un molo può servire una barca sola, oppure la 1 e poi la 3, oppure la 2 e poi la 3. Il turno $\{1, 2\}$ **non c'è**: le barche 1 e 2 sono incompatibili. L'incompatibilità è già sistemata quando costruisco $\mathcal{F}$, sui dati, e nel modello non compare più.

I moli sono tutti uguali: non serve dire **quale** molo fa un turno, basta sapere quali turni vengono fatti.

Variabili: una per turno,

$$y_S = \begin{cases} 1 & \text{se uso un molo con il gruppo di barche } S \in \mathcal{F} \\ 0 & \text{altrimenti} \end{cases}$$

Funzione obiettivo: ogni turno scelto occupa un molo, quindi i moli usati sono $\sum_{S \in \mathcal{F}} y_S$, da minimizzare.

Vincoli: ogni barca deve essere servita, e una volta sola. La barca 1 compare nei turni S_1 e S_4 : fra questi due ne devo scegliere esattamente uno. Stesso discorso per le altre:

$$y_1 + y_4 = 1 \quad (\text{barca 1}) \quad y_2 + y_5 = 1 \quad (\text{barca 2}) \quad y_3 + y_4 + y_5 = 1 \quad (\text{barca 3})$$

In generale, per ogni barca i sommo le variabili dei soli turni che la contengono:

$$\begin{aligned} \min z &= \sum_{S \in \mathcal{F}} y_S && \leftarrow \text{numero di moli usati} \\ \sum_{S \in \mathcal{F}: i \in S} y_S &= 1 \quad \forall i \in B && \leftarrow \text{ogni barca sta in un solo turno scelto} \\ y_S &\in \{0, 1\} && \forall S \in \mathcal{F} \end{aligned}$$

Una soluzione è $y_2 = y_4 = 1$ e le altre a 0: un molo serve la barca 2, un altro serve la 1 e poi la 3. Due moli.

Lo stesso vincolo si scrive in modo più comodo con un **parametro binario**. Numero i turni con un insieme di indici $P = \{1, \dots, 5\}$ (y_p al posto di y_S) e definisco

$$a_{ip} = \begin{cases} 1 & \text{se la barca } i \in B \text{ è nel turno } p \in P \\ 0 & \text{altrimenti} \end{cases}$$

a_{ip}	$p = 1$	$p = 2$	$p = 3$	$p = 4$	$p = 5$
$i = 1$	1	0	0	1	0
$i = 2$	0	1	0	0	1
$i = 3$	0	0	1	1	1

Ogni colonna è un turno, ogni riga una barca. Il vincolo della barca i si legge sulla riga i : moltiplico ogni y_p per il numero sopra e sommo. Dove c'è 0 la variabile sparisce dalla somma, dove c'è 1 resta.

Riga 1: $1y_1 + 0y_2 + 0y_3 + 1y_4 + 0y_5 = 1$, cioè $y_1 + y_4 = 1$ come prima.

$$\min z = \sum_{p \in P} y_p \quad \text{s.t.} \quad \sum_{p \in P} a_{ip} y_p = 1 \quad \forall i \in B, \quad y_p \in \{0, 1\} \quad \forall p \in P$$

Parametro o variabile? a_{ip} è un **parametro**: un dato, come gli orari delle barche. Una volta elencati i turni so già dove vale 1 e dove 0, non c'è niente da decidere, e infatti non compare fra le variabili del modello (non gli si scrive il dominio). x_{im} nel modello di prima era una **variabile**: a quale molo va la barca lo decide il solutore.

Il parametro è anche più flessibile della scrittura con gli insiemi: può valere 2, 3, ... per dire «quante volte», non solo «sì o no». Tornerà utile.

I due modelli a confronto: con le x_{im} ho poche variabili (barche $\times$ moli) e molti vincoli; con i turni ho **pochissimi vincoli** (uno per barca) ma **tantissime variabili**, una per ogni gruppo compatibile, e i gruppi possono essere un numero esponenziale. Questi modelli funzionano molto bene quando le variabili si generano un po' alla volta invece che tutte insieme, ma è una tecnica che nel corso non si vede.

Scegliere turni in modo che ogni barca stia in **esattamente uno** vuol dire dividere B in gruppi che non si sovrappongono e non lasciano fuori nessuno: una **partizione** di B . Cambiando il segno del vincolo si ottengono tre problemi con un nome:

problema	per ogni $i \in B$	ogni elemento sta in...	per le barche
partizione set partitioning	$\sum_{S \in \mathcal{F}: i \in S} y_S = 1$	esattamente un insieme scelto	ogni barca a un solo molo
copertura set covering	$\sum_{S \in \mathcal{F}: i \in S} y_S \geq 1$	almeno un insieme scelto	ogni barca ad almeno un molo
riempimento set packing	$\sum_{S \in \mathcal{F}: i \in S} y_S \leq 1$	al più un insieme scelto	ogni barca al più a un molo

I nomi sono nomenclatura: il prof dice che non li chiederà («e poi mi smentirò»). Servono come campanello: se un problema è una partizione, una copertura o un riempimento, il modello c'è già.

Selezione di una configurazione: l'home restaurant. All'esame e al compitino l'esercizio di modellazione parte da un **testo**, come questo. Tommaso ha un home restaurant e sabato sera ha una prenotazione. La mattina va al mercato con **16,5 €**. Gli ingredienti si vendono solo in **casse** già fatte, che non si possono dividere:

Cassa Proteica (M) 6,5 € 1 filetto M_f 1 avanzo / ossa M_a	Cassa Orto (O) 4,0 € 1 foglie miste O_i 1 radici O_r	Barattolo Salse (S) 1,5 € 1 salsa S
---	---	---

Deve decidere **quante casse comprare** e **quali piatti mettere nel menù**. Ogni piatto usa alcuni ingredienti (una porzione ciascuno) e ha un **valore** v_c , cioè quanto piace ai clienti. Tommaso vuole il menù di valore totale massimo. In più, se il menù ha **almeno 4 piatti** guadagna un **bonus di 10 punti** (menù vario).

c	piatto	M_f	M_a	O_i	O_r	S	valore v_c
1	Tagliata con insalata	1	1	1	0	0	6
2	Stufato rustico	1	1	0	0	1	3
3	Filetto al sale	1	0	0	0	0	2
4	Insalata di carne	1	0	1	0	1	5
5	Filetto con contorno	1	1	1	1	0	9
6	Stufato classico	1	1	0	0	0	10
7	Salsa del giorno	0	0	0	0	1	7
8	Piatto gourmet	1	1	1	1	1	8

La tabella piatto-ingrediente è una matrice di 0 e 1: è di nuovo un parametro binario.

Variabili. Le decisioni sono tre:

- quali piatti fare: $x_c \in \{0, 1\}$ per ogni piatto $c \in C = \{1, \dots, 8\}$, vale 1 se il piatto c è nel menù;

- quante casse comprare: y_M, y_O, y_S . Di una cassa se ne può comprare più di una, quindi non bastano variabili binarie: sono **interi**, $y \in \mathbb{N}_0$ (i numeri naturali con lo zero: 0 non la compro, 1 ne compro una, 2 ne compro due...). È un'estensione delle binarie: dicono **se** faccio una cosa e **quante volte**;
- se prendo il bonus: $z \in \{0, 1\}$, vale 1 se il menù ha almeno 4 piatti.

Funzione obiettivo: il valore di ogni piatto scelto, più il bonus $b = 10$ se $z = 1$.

$$\max w = \sum_{c \in C} v_c x_c + bz \quad \text{cioè} \quad \max 6x_1 + 3x_2 + 2x_3 + 5x_4 + 9x_5 + 10x_6 + 7x_7 + 8x_8 + 10z$$

Vincolo del bonus. Così com'è, z è slegata dal resto: il max la metterebbe sempre a 1 e prenderebbe il bonus anche con un piatto solo. È la situazione delle y_f con le antenne: serve un vincolo che leghi z alle x_c . Deve impedire $z = 1$ quando i piatti sono meno di 4:

$$\sum_{c \in C} x_c \geq 4z \quad (\text{o, che è lo stesso, } z \leq \frac{1}{4} \sum_{c \in C} x_c)$$

Con $z = 0$ dice $\sum x_c \geq 0$, sempre vero. Con $z = 1$ dice $\sum x_c \geq 4$: posso avere $z = 1$ solo con almeno 4 piatti. Visto dall'altra parte: con 3 piatti $z \leq \frac{3}{4}$, e una binaria $\leq \frac{3}{4}$ può solo valere 0.

E quando i piatti sono 4 o più? Il vincolo permette sia $z = 0$ sia $z = 1$. Ma z ha coefficiente positivo (+10) in una funzione obiettivo di **massimo**: appena può, il modello la mette a 1. **Basta questo vincolo** perché la funzione obiettivo spinge dalla parte giusta.

Non sempre si è così fortunati. Se z non comparisse nella funzione obiettivo (o se il suo coefficiente spingesse dalla parte sbagliata), niente la obbligherebbe a valere 1 con 4 piatti o più. Per avere « $z = 1$ se e solo se i piatti sono almeno 4» servirebbe un secondo vincolo, che obbliga z a 1:

$$\sum_{c \in C} x_c \leq 3 + (|C| - 3)z$$

($|C| = 8$ è il numero di piatti.) Con $z = 0$ dice $\sum x_c \leq 3$: per fare 4 piatti o più devo per forza avere $z = 1$. Con $z = 1$ dice $\sum x_c \leq |C|$, sempre vero. La tabella mostra, per ogni numero di piatti nel menù, quali valori di z lascia passare ciascun vincolo:

piatti nel menù	0	1	2	3	4	5	6	7	8
$\sum x_c \geq 4z$	0	0	0	0	0 o 1	0 o 1	0 o 1	0 o 1	0 o 1
$\sum x_c \leq 3 + 5z$	0 o 1	0 o 1	0 o 1	0 o 1	1	1	1	1	1
tutti e due	0	0	0	0	1	1	1	1	1

Il primo vincolo è del tipo «**posso** solo se» (tiene z a 0), il secondo del tipo «**devo** se» (spinge z a 1): lo stesso schema delle relazioni logiche.

Vincoli sugli ingredienti. I piatti scelti non possono usare più ingredienti di quelli comprati. Ogni Cassa Proteica contiene un filetto, quindi i filetti disponibili sono y_M ; quelli che servono sono tanti quanti i piatti scelti che usano il filetto. Un vincolo per ogni ingrediente, letto dalle colonne della tabella:

$$\begin{array}{ll} \text{filetto } M_f: & x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_8 \leq y_M \\ \text{avanzo / ossa } M_a: & x_1 + x_2 + x_5 + x_6 + x_8 \leq y_M \\ \text{foglie } O_i: & x_1 + x_4 + x_5 + x_8 \leq y_O \\ \text{radici } O_r: & x_5 + x_8 \leq y_O \\ \text{salsa } S: & x_2 + x_4 + x_7 + x_8 \leq y_S \end{array}$$

Se faccio i piatti 5 e 6, tutti e due con il filetto, il primo vincolo dà $y_M \geq 2$: due casse proteiche. Può capitare di comprare ingredienti che poi non uso (con le casse arrivano anche quelli).

Scrivere cinque vincoli a mano va bene qui, ma non con cento ingredienti. La stessa cosa si scrive in forma generale in due modi. Con gli **insiemi**: chiamo $H = \{M_f, M_a, O_i, O_r, S\}$ gli ingredienti e C_h l'insieme dei piatti che usano l'ingrediente h ; per il filetto, $\sum_{c \in C_{M_f}} x_c \leq y_M$. Oppure con il **parametro binario** a_{hc} , che vale 1 se il piatto c usa l'ingrediente h (è la tabella dei piatti): $\sum_{c \in C} a_{M_f, c} x_c \leq y_M$. La somma è su tutti i piatti, e quelli che non usano il filetto spariscono perché moltiplicati per 0.

Il parametro permette di arrivare a **un solo vincolo per tutti gli ingredienti**. Chiamo $J = \{M, O, S\}$ i tipi di cassa e aggiungo un secondo parametro, $d_{hj} = 1$ se l'ingrediente h sta nella cassa j :

$$\underbrace{\sum_{c \in C} a_{hc} x_c}_{\text{quanto ne serve}} \leq \underbrace{\sum_{j \in J} d_{hj} y_j}_{\text{quanto ne compro}} \quad \forall h \in H$$

Questa forma regge anche casi che le altre non reggono, cambiando solo i numeri nei parametri: un piatto che usa **due** porzioni dello stesso ingrediente ($a_{hc} = 2$), una cassa che contiene **più unità** di un ingrediente ($d_{hj} = 2$), lo stesso ingrediente presente in **casse diverse** (più d_{hj} a 1 sulla stessa riga). All'esame va bene anche la forma lunga.

Vincolo di budget, come nello zaino: con k_j costo della cassa j e $B = 16.5$,

$$\sum_{j \in J} k_j y_j \leq B \quad \text{cioè} \quad 6.5y_M + 4y_O + 1.5y_S \leq 16.5$$

Il $\leq$ perché posso spendere anche meno. Non serve una variabile in più per «quanto spendo»: è già la somma a sinistra.

Il modello completo, con i domini (che vanno sempre scritti: il solutore non sa da solo che z è binaria):

$$\begin{aligned} \max w &= \sum_{c \in C} v_c x_c + bz && \leftarrow \text{valore del menù più bonus} \\ \sum_{c \in C} x_c &\geq 4z && \leftarrow \text{bonus solo con almeno 4 piatti} \\ \sum_{c \in C} a_{hc} x_c &\leq \sum_{j \in J} d_{hj} y_j && \forall h \in H \quad \leftarrow \text{ingredienti usati} \leq \text{comprati} \\ \sum_{j \in J} k_j y_j &\leq B && \leftarrow \text{budget} \\ x_c &\in \{0, 1\} \quad \forall c \in C, \quad y_j \in \mathbb{N}_0 \quad \forall j \in J, \quad z \in \{0, 1\} \end{aligned}$$

Instradamento: una variabile per ogni cammino. Nei moli la famiglia $\mathcal{F}$ si poteva elencare guardando gli orari. Spesso la famiglia non è data: va costruita dalle regole del testo. La Trans-Port, azienda di logistica, deve pianificare i viaggi dei suoi camion. Ogni camion parte dal **deposito** d , **consegna** container pieni agli **importatori** (insieme I) e/o **ritira** container vuoti dagli **esportatori** (insieme E), poi torna a d . Le regole:

- un camion porta **al massimo due container**;
- **prima consegna, poi ritira**: mai un esportatore prima di un importatore;
- si conosce la distanza c_{hk} fra ogni coppia di punti (deposito, importatori, esportatori), uguale nei due versi;
- ogni importatore i ha una domanda q_i di container da ricevere, ogni esportatore e una domanda q_e di container da far ritirare;
- i camion sono quanti ne servono (un camion rientrato può ripartire).

Si vogliono soddisfare tutte le domande percorrendo la **minima distanza totale**.

Con due container al massimo e la regola «prima consegna, poi ritiro», i giri possibili sono pochi: al più 2 fermate dagli importatori seguite da al più 2 dagli esportatori. Si dividono per numero di fermate:

insieme	cammini	perché non ce ne sono altri
P_1 1 fermata		
P_2 2 fermate		manca $d \rightarrow e \rightarrow i \rightarrow d$: prima si consegna, poi si ritira
P_3 3 fermate		mai tre importatori o tre esportatori: il camion porta due container
P_4 4 fermate		parte con 2 pieni, torna con 2 vuoti

L'insieme di tutti i cammini è $P = P_1 \cup P_2 \cup P_3 \cup P_4$. Da qui in poi tutto il lavoro è sui dati, prima del modello:

- il **costo** c_p del cammino p è la somma delle distanze dei suoi tratti. Per il cammino $d \rightarrow i \rightarrow e \rightarrow d$ è $c_{di} + c_{ie} + c_{ed}$. È un parametro: lo calcolo una volta generati i cammini;
- nulla vieta $i_1 = i_2$ (o $e_1 = e_2$): lo stesso camion porta **due container allo stesso cliente**, e il tratto fra le due «fermate» costa 0. Per questo il parametro che dice se un cammino passa da un cliente non è binario: a_{pi} = numero di volte (0, 1 o 2) che il cammino p visita l'importatore i , e a_{pe} lo stesso per l'esportatore e . Ecco il caso in cui il parametro vale 2;
- se un cliente è sia importatore sia esportatore, lo si tratta come due punti diversi a distanza 0.

Variabili: y_p = numero di camion che fanno il cammino $p \in P$. È intera e non binaria perché lo stesso giro può servire più volte (un importatore che aspetta 6 container riceve più camion).

$$\begin{aligned}
 \min \sum_{p \in P} c_p y_p & \quad \leftarrow \text{distanza totale} \\
 \sum_{p \in P} a_{pi} y_p &= q_i \quad \forall i \in I \quad \leftarrow \text{ogni importatore riceve i suoi container} \\
 \sum_{p \in P} a_{pe} y_p &= q_e \quad \forall e \in E \quad \leftarrow \text{da ogni esportatore ritiro i suoi container} \\
 y_p &\in \mathbb{N}_0 \quad \forall p \in P
 \end{aligned}$$

I vincoli hanno la forma della partizione, con q_i al posto di 1: sommo i cammini che passano da i , ciascuno contato quanti container gli lascia, e il totale deve essere la sua domanda. La capacità del camion e l'ordine consegna-ritiro **non compaiono nel modello**: sono già dentro la costruzione di P , come l'incompatibilità delle barche era dentro $\mathcal{F}$.

Tutta la difficoltà sta nel generare i cammini; il modello poi è semplice. Qui si può fare perché i cammini hanno al più 4 fermate. Se i camion potessero fare giri lunghi i cammini sarebbero un numero enorme

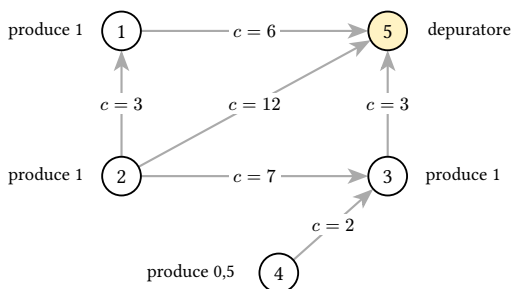
(cresce come il fattoriale, $n! = n \cdot (n-1) \dots 2 \cdot 1$, il numero di modi di mettere in ordine n fermate): esistono algoritmi che li generano solo quando servono (generazione di colonne), ma nel corso non si vedono.

1.6 Problemi di flusso su rete

Finora le variabili erano **binarie** (faccio / non faccio) o **interi** (quante volte lo faccio). Anche una variabile **continua**, cioè un numero reale qualunque, può portare un'informazione logica: se vale 0 quella scelta non viene fatta, se è positiva la scelta è fatta e il valore dice **quanto**. Una sola variabile fa da interruttore e da quantità.

Questa idea si usa soprattutto nei **problemi di flusso su rete**. Una rete è un grafo orientato: qualcosa (merce, energia, dati, acqua, gas) si muove lungo gli archi, e le variabili dicono **quanto** ne passa su ogni arco. Di conseguenza dicono anche quali archi vengono usati davvero.

Il problema delle fognature. Una città ha 4 quartieri, i vertici $V = \{1, 2, 3, 4\}$. Ogni quartiere produce una quantità nota di acque reflue, in m^3/h : 1 i quartieri 1, 2 e 3, e 0,5 il quartiere 4. Tutta l'acqua va portata a un unico depuratore, il nodo 5. Le condotte che si possono costruire sono gli archi $A = \{(1, 5), (2, 1), (2, 3), (2, 5), (3, 5), (4, 3)\}$: ogni condotta ha un verso. Il costo della condotta (i, j) è proporzionale all'acqua che ci passa: c_{ij} per ogni m^3/h . Si vuole decidere quali condotte costruire e quanto grandi, per portare tutto al depuratore al costo minimo. È un problema di **disegno di rete**.



Variabili: x_{ij} = flusso (m^3/h) sulla condotta $(i, j) \in A$. È continua. Se $x_{ij} = 0$ la condotta non si costruisce; se è positiva si costruisce, e il valore è anche la sua **dimensione**.

Funzione obiettivo: ogni condotta costa c_{ij} per ogni unità di flusso,

$$\min \sum_{(i,j) \in A} c_{ij} x_{ij}$$

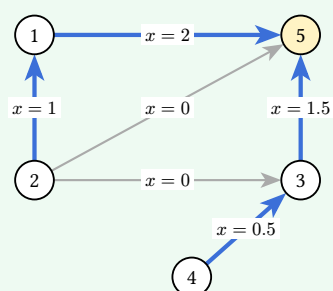
$$\text{cioè } \min 6x_{15} + 3x_{21} + 7x_{23} + 12x_{25} + 3x_{35} + 2x_{43}.$$

Vincoli. Senza vincoli il minimo sarebbe tutto a 0: niente condotte, costo zero. Bisogna dire dove va l'acqua. La regola è la **conservazione del flusso**: in ogni nodo l'acqua non si crea e non sparisce, quindi **tutto quello che entra è uguale a tutto quello che esce**. Quello che il quartiere produce conta come acqua che entra nel nodo.

nodo	entra		esce
1	$x_{21} + 1$	=	x_{15}
2	1	=	$x_{21} + x_{23} + x_{25}$
3	$x_{23} + x_{43} + 1$	=	x_{35}
4	0.5	=	x_{43}
5	$x_{15} + x_{25} + x_{35}$	=	$1 + 1 + 1 + 0.5 = 3.5$

Il nodo 4 non ha scelta: il suo 0,5 va tutto sulla condotta (4, 3). Il nodo 2 invece ha tre strade e deve dividere la sua unità fra x_{21} , x_{23} e x_{25} : è lì che il modello decide. Nel nodo 5 quello che «esce» è l'acqua che il depuratore assorbe: tutta quella prodotta, 3,5. Questo vincolo è **ridondante**: si ottiene sommando gli altri quattro.

Da sapere — una soluzione di esempio (aggiunto, non spiegato in aula)



Per vedere i vincoli all'opera, una soluzione che li rispetta tutti. In blu le condotte con flusso positivo: sono quelle da costruire; le grigie hanno $x = 0$ e non si costruiscono.

Controllo sul nodo 1: entra $x_{21} + 1 = 2$, esce $x_{15} = 2$. Sul nodo 3: entra $x_{23} + x_{43} + 1 = 0 + 0.5 + 1$, esce $x_{35} = 1.5$.

Costo: $3 \cdot 1 + 6 \cdot 2 + 2 \cdot 0.5 + 3 \cdot 1.5 = 20.5$.

Per scrivere questi vincoli in forma generale si dà un nome alla quantità che ogni nodo produce o richiede: il **deficit** b_i , cioè quanto **manca** al nodo i . Per ragioni storiche si ragiona «al contrario»: un nodo che produce ha deficit **negativo**.

deficit	il nodo è...	nelle fognature
$b_i < 0$	una sorgente : offre $ b_i $ unità di flusso	i quartieri: $b_1 = b_2 = b_3 = -1$, $b_4 = -0.5$
$b_i > 0$	un pozzo : richiede b_i unità di flusso	il depuratore: $b_5 = 3.5$
$b_i = 0$	un nodo di transito : il flusso ci passa e basta	nessuno (sarebbe un incrocio di tubi)

Portando tutte le variabili a sinistra, «entra = esce» diventa **entra - esce = deficit**. Sono i **vincoli di conservazione del flusso** (o di bilancio), uno per nodo:

$$\sum_{j:(j,i) \in A} x_{ji} - \sum_{j:(i,j) \in A} x_{ij} = b_i \quad \forall i \in V$$

La prima somma è sugli archi che **entrano** in i , la seconda su quelli che **escono**. Per il nodo 1: $x_{21} - x_{15} = -1$, che è $x_{21} + 1 = x_{15}$ di prima.

Tutto quello che le sorgenti offrono deve essere assorbito dai pozzi, quindi i deficit sommati fanno zero: $\sum_i b_i = 0$. Il deficit del depuratore non è un dato in più: $b_5 = -(b_1 + b_2 + b_3 + b_4) = 3.5$.